Module 3

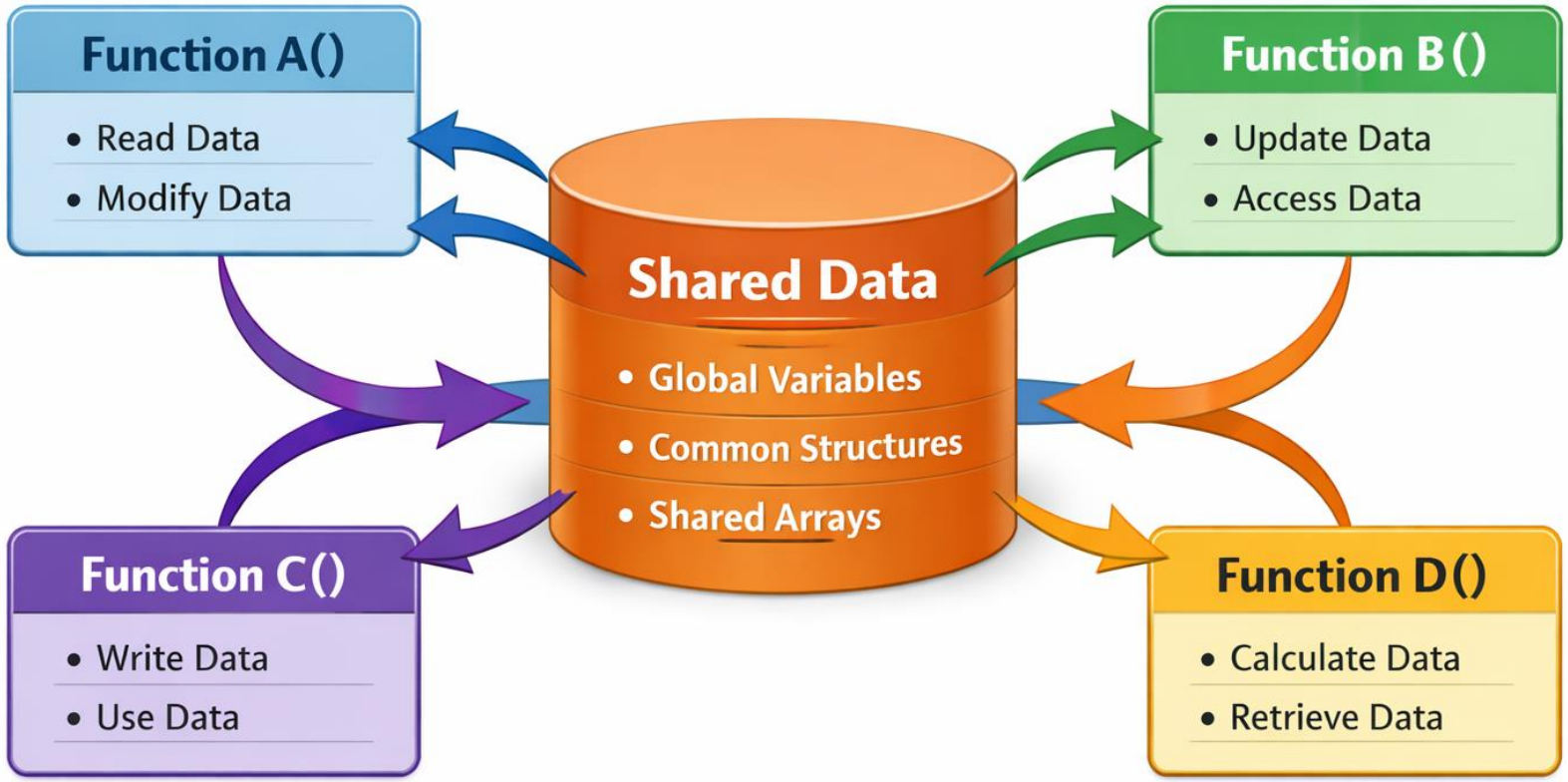
Introduction to Object Oriented Programming

Programming before OOP

- Programs written as a collection of functions
- Data shared between multiple functions
- Any function can modify any data
- Works well only for small programs
- Becomes difficult as program size increases

Procedural Programming

Multiple Functions Accessing Shared Data

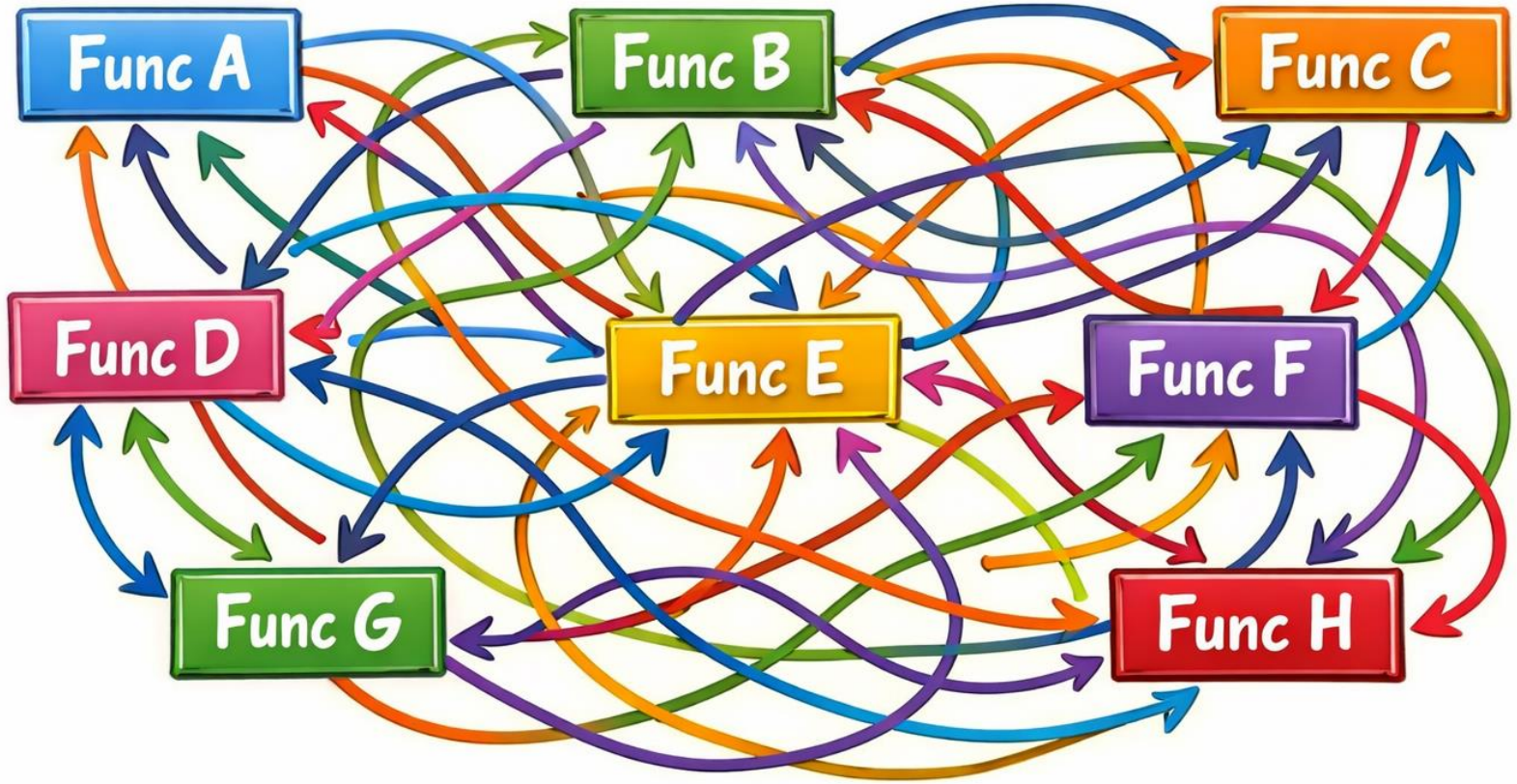


Problems with Large C Programs

- No proper data protection
- Difficult to track data flow
- Debugging becomes complex
- Code reuse is minimal
- Changes affect many parts of the program

"Spaghetti Code"

Tangled Function Calls



Real World vs Procedural Programs

REAL WORLD CONSISTS OF ENTITIES

EACH ENTITY HAS DATA AND ACTIONS

PROCEDURAL CODE FOCUSES ONLY ON ACTIONS

REAL-WORLD MAPPING BECOMES UNCLEAR

Real-World Objects

- Fixed & Specific
- Predefined Attributes

Procedural Functions

- Generate Car()
- Create House()
- Random Apple()
- MakeChair(x, y)

- Dynamic & Flexible
- Generated on Demand

Need for Object-Oriented Programming

PROGRAMS SHOULD MODEL REAL-WORLD ENTITIES

DATA AND ACTIONS SHOULD STAY TOGETHER

CONTROLLED ACCESS TO DATA IS REQUIRED

LEADS TO BETTER ORGANIZATION

What is Object-Oriented Programming

Programming style based on objects

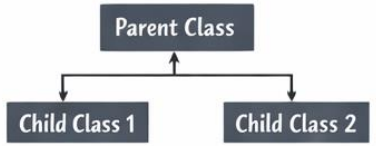
Objects combine data and functions

Programs are built using objects

Improves structure and readability

Object-Oriented Programming

Encapsulation



Inheritance

Polymorphism

Abstraction

Class

Car Class
Attributes: color, speed
Methods: drive(), honk()

Defines a Blueprint

Attributes
Properties / Data

Methods
Functions / Behaviors

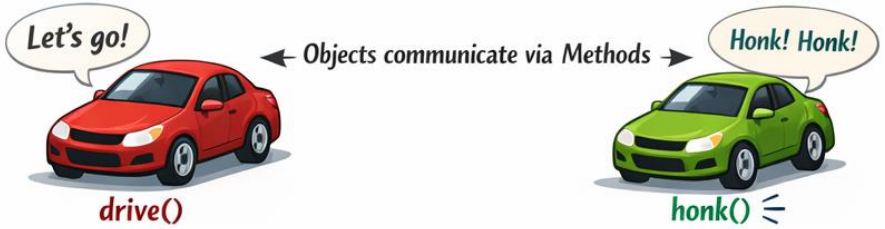
Objects

Red Car Blue Car Green Car

Instances of the Class

Individual Objects

MESSAGE PASSING:



Object – Basic Idea

Object represents a real-world entity

Has data (state)

Has functions (behavior)

Example: Student, Car, BankAccount

Objects:

– State as Attributes & Behaviors as Methods

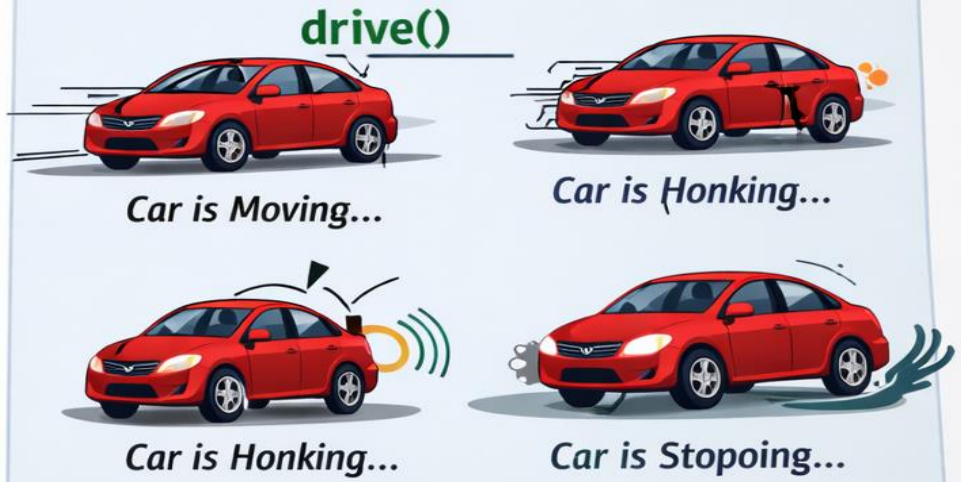
State as Attributes

Describes what an object is

Attributes define the **properties**

Behaviors as Methods

*Describes what an object **does***



Methods *define the* **actions**

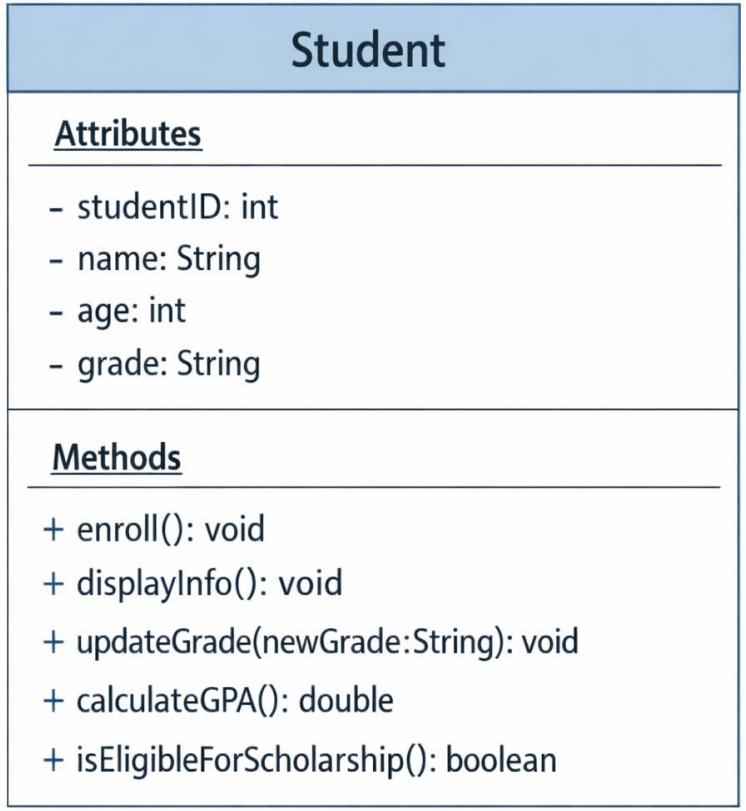
Analogy: Attributes are the "nouns." Methods are the "verbs."

Object

State	Behavior
• attribute1 • attribute2 • attribute3	• method1() • method2() • method3() • method4()

Example of an Object (Student)

- Student is an object
- Data: roll number, name, marks
- Functions: display(), calculateGrade()



Class – Basic Idea

- Class is a blueprint
- Used to create objects
- Defines data and functions
- Does not occupy memory

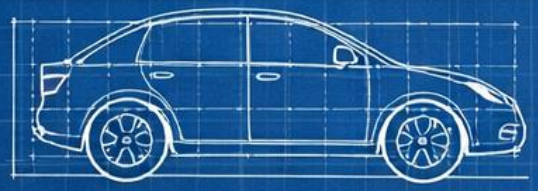
Class Blueprint Analogy

Class vs. Objects

Class (Blueprint)

Car

- Attributes:
  - color
  - model
  - speed
- Methods:
  - drive()
  - brake()



Template / Design Plan

Objects (Instances)

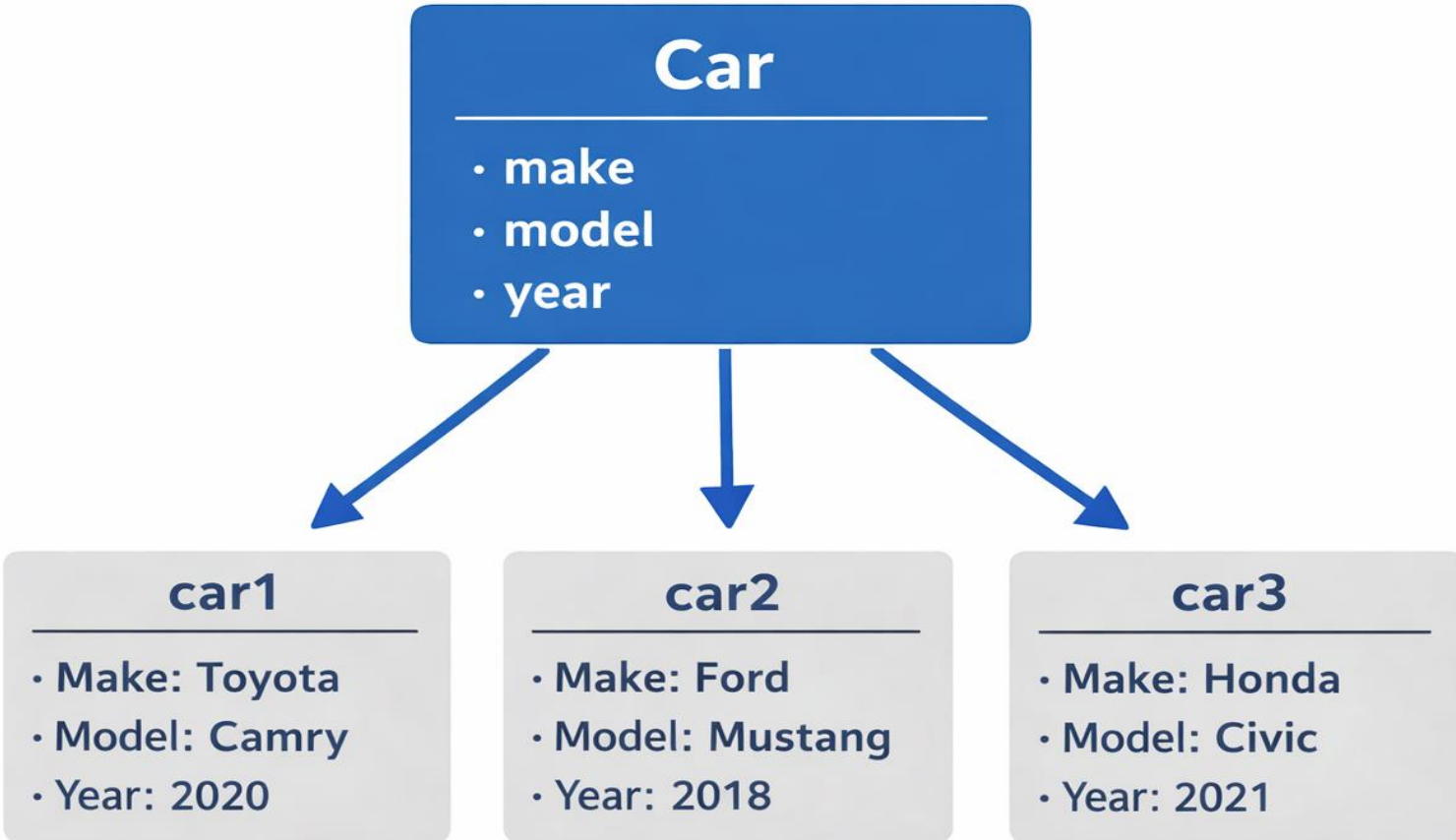
car1:
Color: Red
Model: Sports Car
Speed: 150 mph

car2:
Color: Blue
Model: Sedan
Speed: 100 mph

car3:
Color: Yellow
Model: SUV
Speed: 80 mph

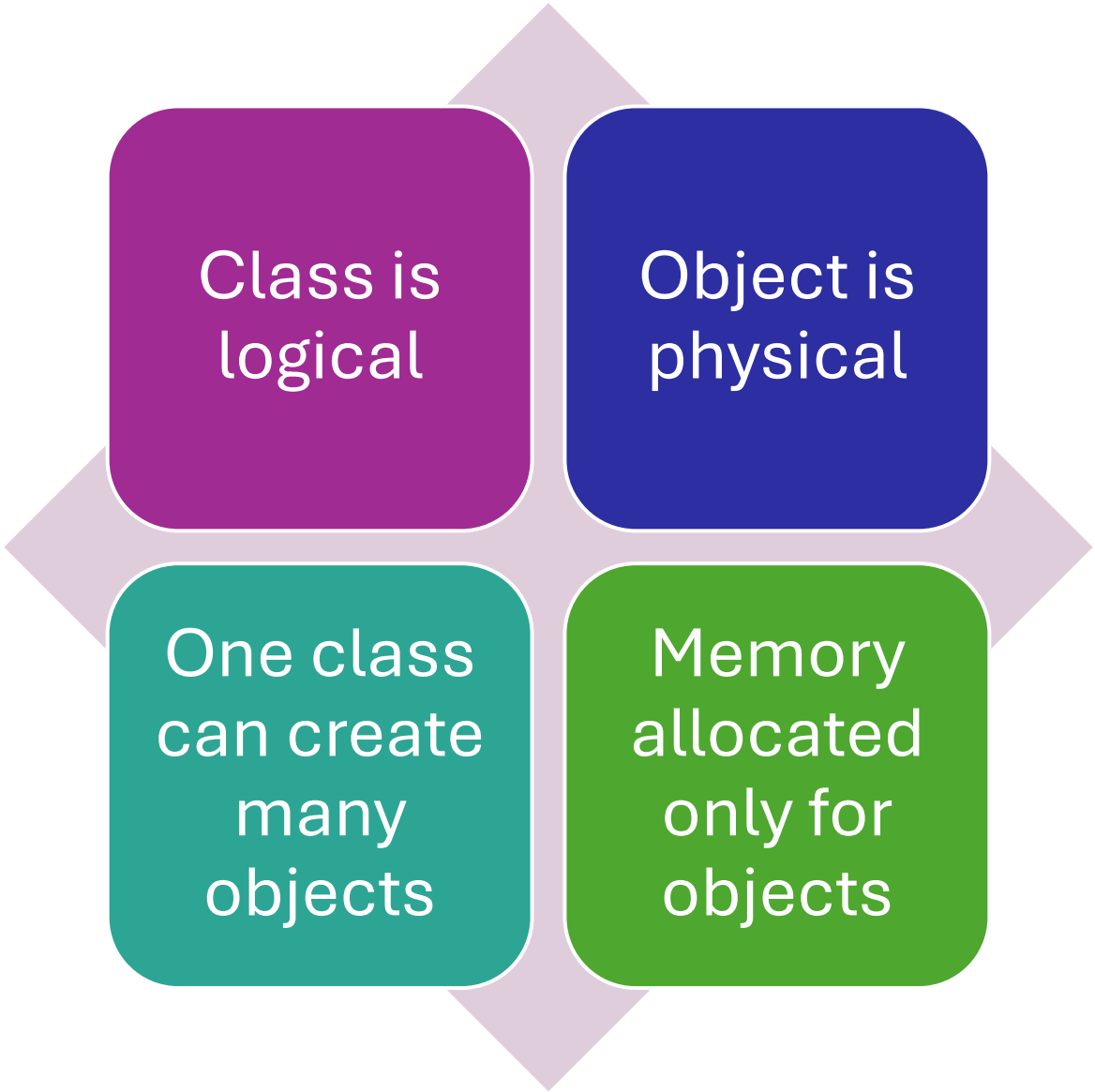
Objects Created from the Class

Class is the Blueprint, Objects are the Instances



One Class, Multiple Objects

Class vs Object



Why We Need Class and Object

Natural representation of real-world problems

Better organization of code

Foundation for OOP features

Improves readability and maintenance

Features of Object-Oriented Programming

Encapsulation

Abstraction

Inheritance

Polymorphism

Encapsulation

- Data and functions are wrapped together
- Direct access to data is restricted
- Improves data security
- Prevents misuse

Encapsulation

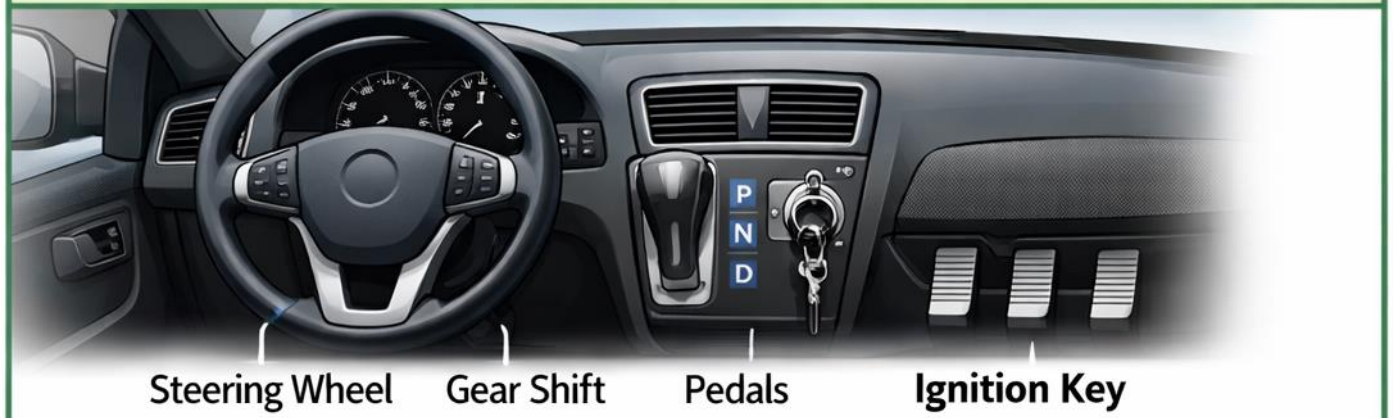
Data Hidden Inside the Object

Abstraction

- Shows only essential details
- Hides internal implementation
- User interacts through interface
- Simplifies usage

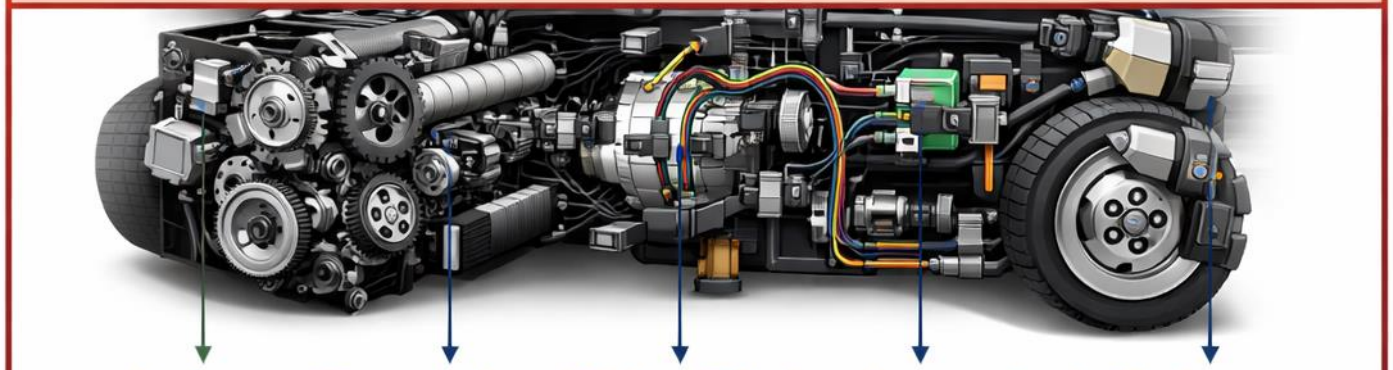
Abstraction in OOP

Interface (What you interact with)



Abstraction

Hidden Details (What you don't see)



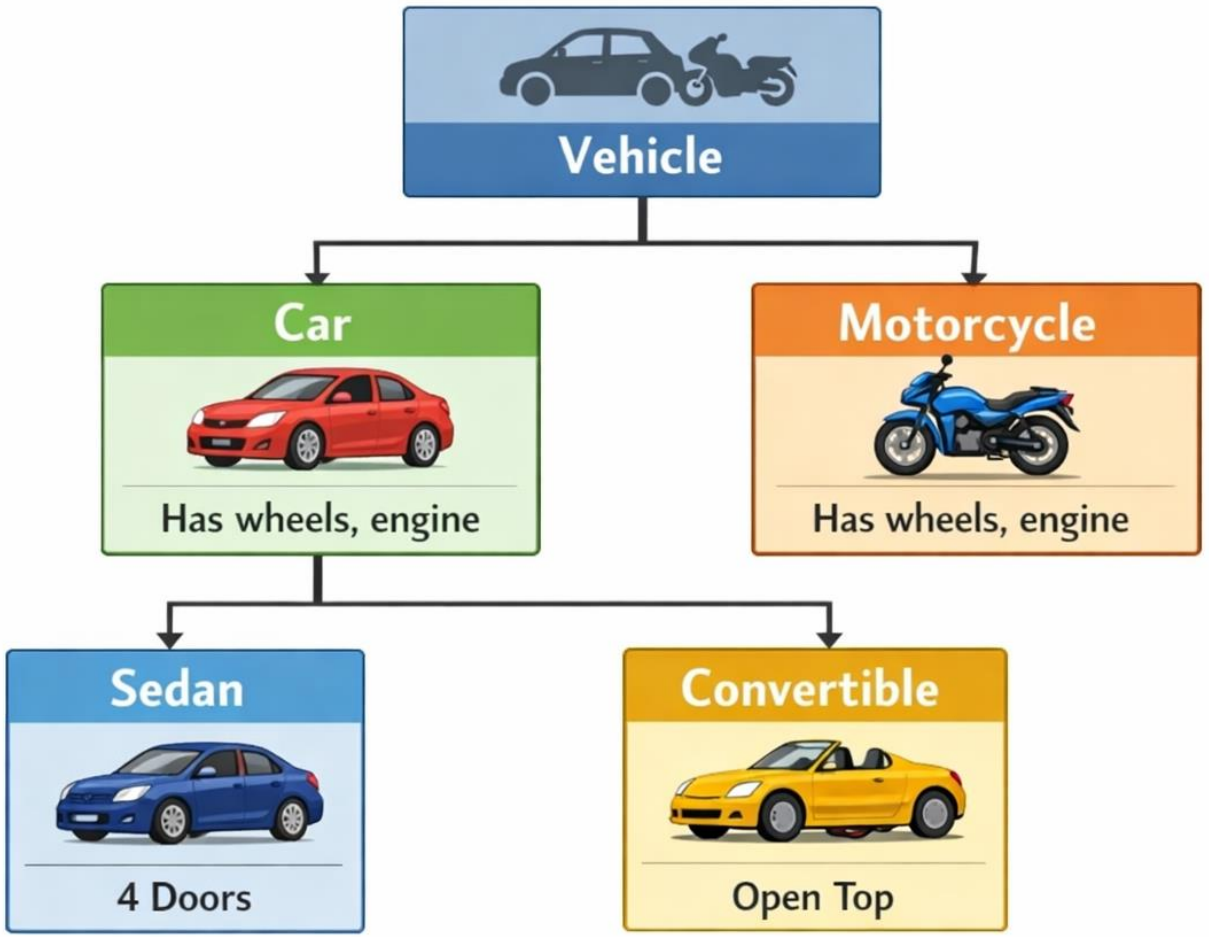
Inheritance

One class acquires properties of another

Promotes code reuse

Forms parent–child relationship

Inheritance Hierarchy - Car Analogy



Polymorphism

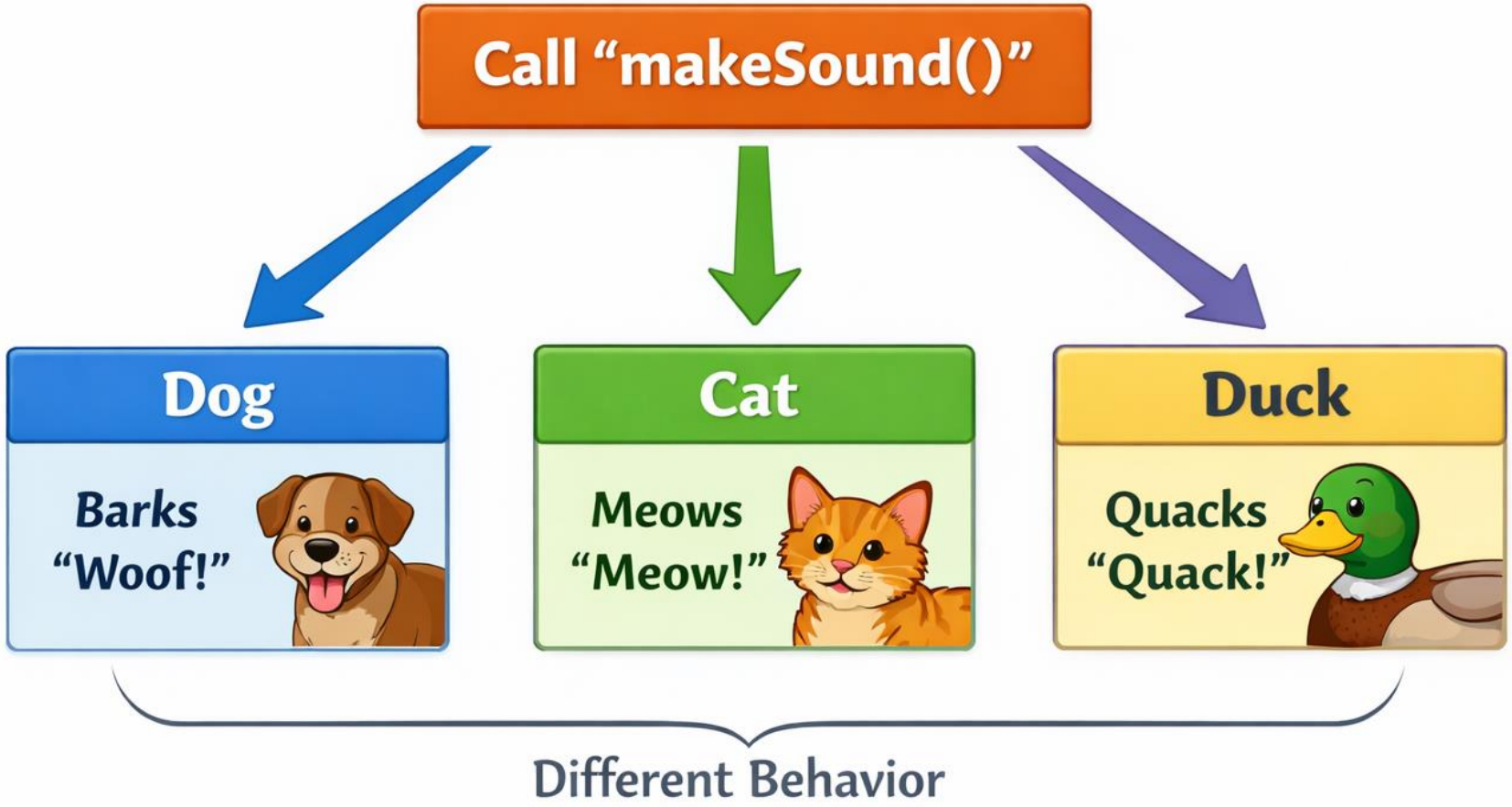
Same function name

Different behavior

Depends on object type

Polymorphism

Same Action, Different Results



Simple C++ Program

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, C++" << endl;
    return 0;
}
```

```
#include <iostream>

int main() {
    std::cout << "Hello World!";
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    cout << "Sum = " << a + b << endl;
    return 0;
}
```

- No format specifiers (%d)
- >> for input, << for output
- Type safety handled by C++

```
#include <iostream>
using namespace std;

int main() {
   int n;
   cout << "Enter a number: ";
   cin >> n;
   for (int i = 1; i <= n; i++) {
      if (i % 2 == 0) {
         cout << i << " is Even" << endl;
      } else {
         cout << i << " is Odd" << endl;
      }   }
   return 0;
}
```

Class

- A class is a user-defined data type
- It groups related data and functions
- Data represents state
- Functions represent behavior
- Fundamental unit of OOP

Class as a Blueprint

CLASS DEFINES STRUCTURE OF OBJECTS

OBJECTS ARE CREATED USING CLASS

MULTIPLE OBJECTS CAN USE SAME CLASS

CLASS ITSELF HAS NO PHYSICAL EXISTENCE

CLASS (BLUEPRINT)

Properties:

- Number of Rooms
- Color
- Has Garage

Methods:

- Open Door()
- Turn On Lights()

INSTANTIATION

OBJECTS (INSTANCES)

House 1

- Rooms: 3
- Color: Red
- Has Garage: Yes

House 2

- Rooms: 4
- Color: Yellow
- Has Garage: No

House 3

- Rooms: 2
- Color: Blue
- Has Garage: Yes

Basic Class Syntax

```
class Student {
    int roll;
    char name[20];
public:
    void display();
};
```

- class keyword starts definition
- Curly braces define class body
- Semicolon is mandatory

Data Members of a Class

Variables declared inside a class

Represent object's state

Each object has its own copy

- int roll;
- float marks;

Member Functions of a Class

Functions declared inside a class

Define object behavior

Can access data members directly

- void display();

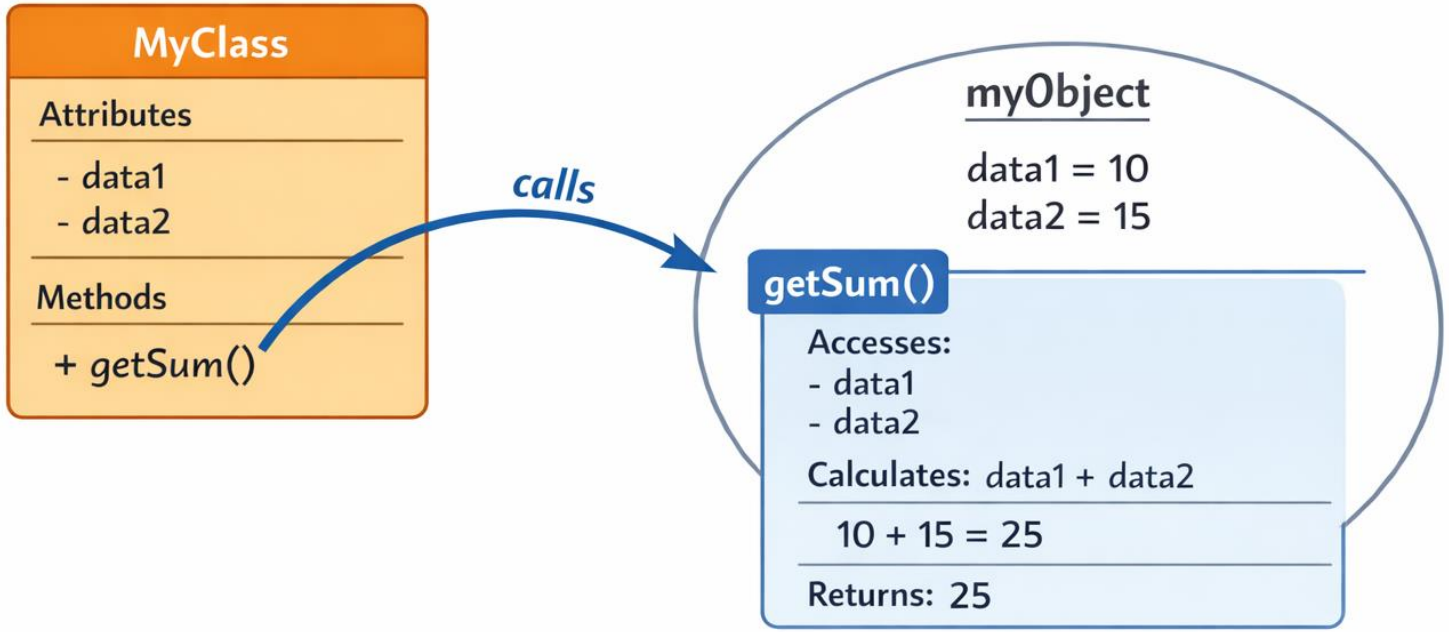
Why Functions Inside a Class

Functions operate on object data

No need to pass data as parameters

Improves cohesion and clarity

Member Function Accessing Internal Data



Access Specifiers – Overview

Control visibility of class members

Decide what can be accessed outside

Support data protection

Types:

public

private

protected

Public and Private Members

public members are accessible outside class

private members are accessible only inside class

Default access in class is private

Class and Memory

Defining a class does not allocate memory

Memory is allocated only when object is created

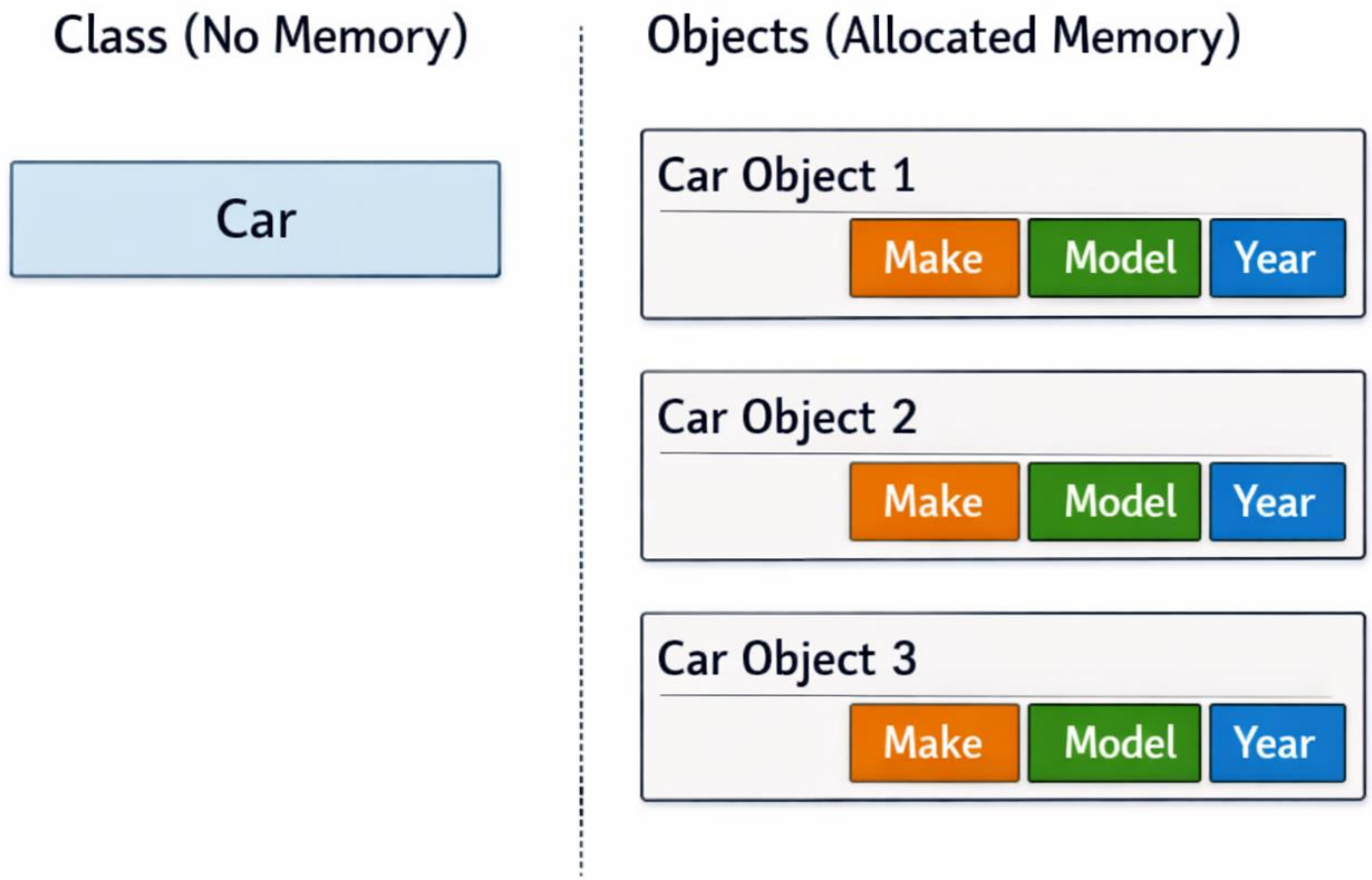
Class is a logical entity

Class vs Object Memory

CLASS → NO MEMORY

OBJECT → MEMORY ALLOCATED

EACH OBJECT HAS SEPARATE MEMORY


Class (No Memory)
Car
Objects (Allocated Memory)
Car Object 1
Make
Model
Year
Car Object 2
Make
Model
Year
Car Object 3
Make
Model
Year

What is an Object

OBJECT IS AN INSTANCE OF A CLASS

REPRESENTS REAL ENTITY IN MEMORY

CREATED USING CLASS NAME

WORKS ON OBJECT DATA

Object Creation in C++

- Objects are created using class name
- Memory allocated at creation time
- Student s1;

Multiple Objects from One Class

- Same class can create many objects
- Each object has independent data
- Member functions are shared
- Student s1, s2;

Accessing Object Members

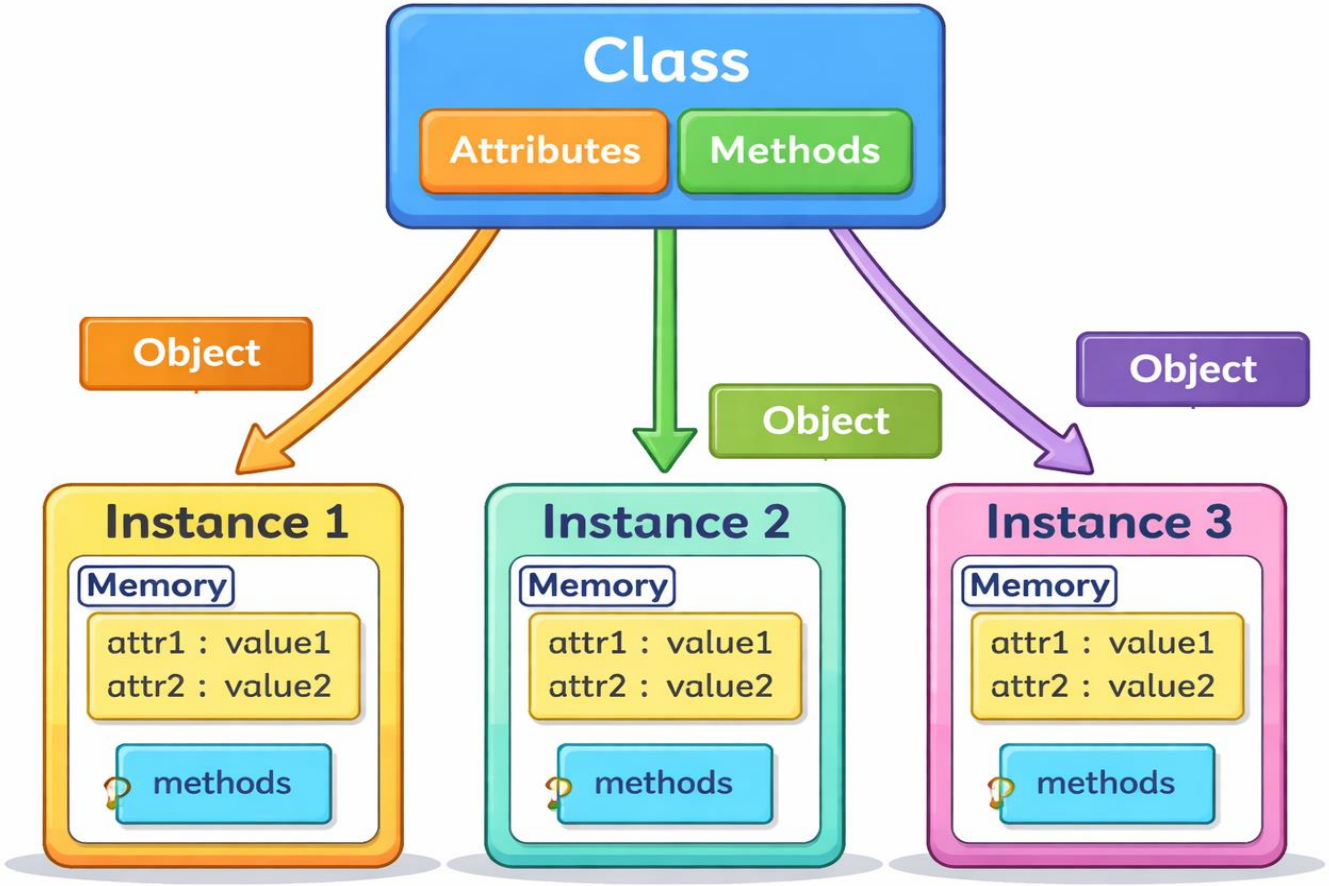
- Dot (.) operator is used
- Used to access data and functions
- s1.roll = 10;
- s1.display();

Calling Member Functions

- Member functions act on object’s data
- Called using object name
- s1.display();
- s2.display();

Object Memory Allocation

- Each object occupies separate memory
- Prevents data interference
- Enables multiple instances



```
#include <iostream>
using namespace std;

class Student {
   int roll;
public:
   void display() {
      cout << "Roll number: " << roll << endl;
   } };

int main() {
   Student s1;
   s1.roll = 10;
   s1.display();
   return 0;
}
```

What is Encapsulation

- Binding data and functions together
- Restricts direct data access
- Data is accessed through functions
- Core principle of OOP

Why Encapsulation is Needed

- Prevents invalid data modification
- Protects internal state
- Improves program reliability
- Reduces debugging effort

Public Members

- Accessible outside the class
- Used as interface to object
- Allow interaction with object

```
public:
    void display();
```

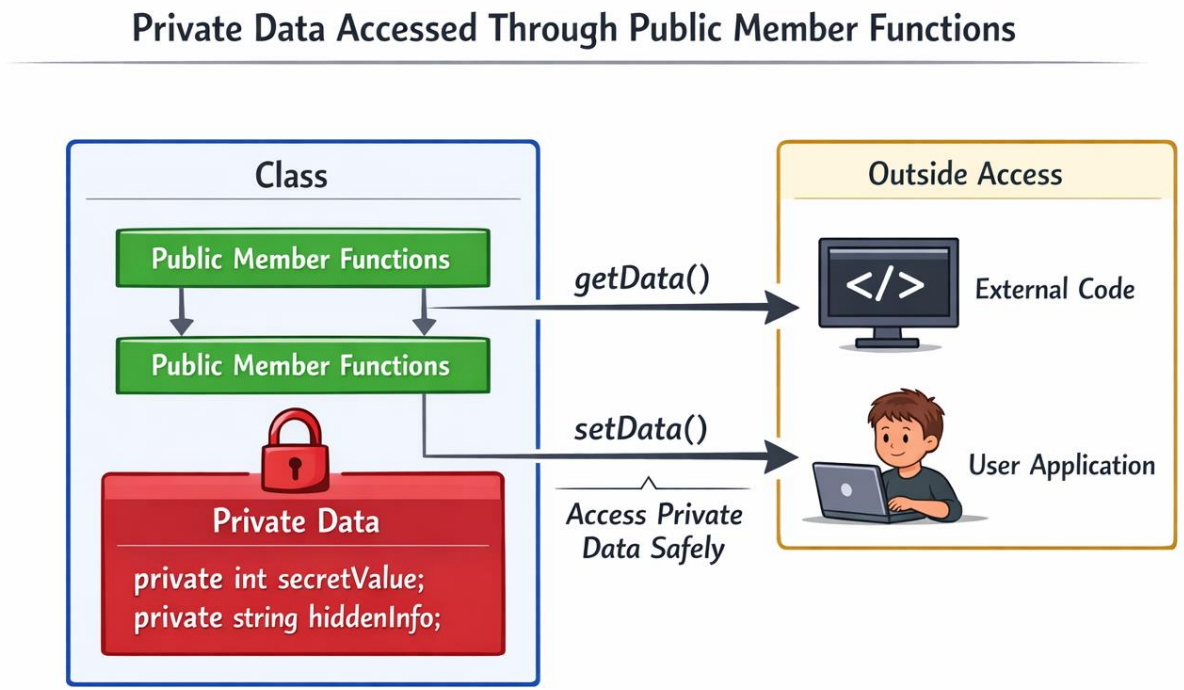
Private Members

- Not accessible outside class
- Protect sensitive data
- Default access type in class

```
private:
  int roll;
```

Controlled Access to Data

- Private data accessed via public functions
- Functions act as gatekeepers
- Validation logic can be added



```
class Student {
private:
   int roll;
public:
   void setRoll(int r);
   void display();
};
```

- Data hidden
- Access provided through functions

```
#include <iostream>
using namespace std;
 class Student {
private:
   int roll;
 public:
   void setRoll(int r) {
      roll = r;
   }
   void display() {
      cout << "Roll number: " << roll << endl;
   }
};
 int main() {
   Student s1;
   s1.setRoll(25);
   s1.display();
   return 0;
}
```

Data is private
Access controlled via functions

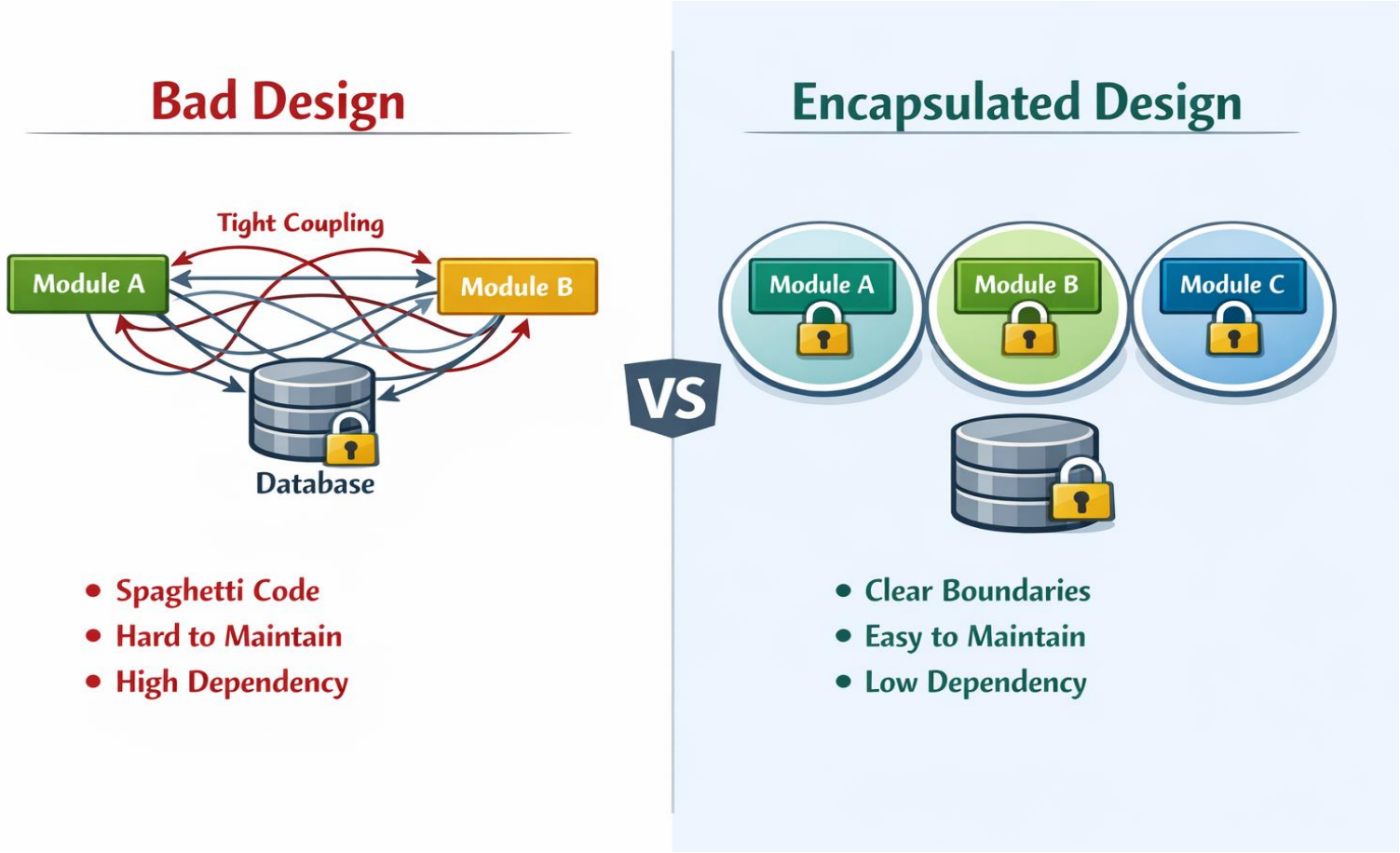
```
#include <iostream>
using namespace std;
class Student {
private:
   int roll;
public:
   void setRoll(int r) {
      if (r > 0)
         roll = r;
      else
         roll = 0;    }
   void display() {
      cout << "Roll number: " << roll << endl;
   }
};
```

```
int main() {
   Student s1;
   s1.setRoll(-5);
   s1.display();
   return 0;
}
```

- *Demonstrates real use of encapsulation*
- *Invalid data is prevented*

Good Design vs Bad Design

- Public data → unsafe
- Private data → controlled
- Encapsulation improves design quality



Data Initialization – Why it is Needed?

- Objects need valid initial values
- Prevents garbage data
- Ensures consistency at creation

What is a Constructor

- Special member function
- Same name as class
- Called automatically
- No return type

Default Constructor

- Takes no parameters
- Initializes default values
- Called automatically

```
Student() {
    roll = 0;
}
```

Parameterized Constructor

- Accepts parameters
- Initializes object with values

```
Student(int r) {
   roll = r;
}
```

Constructor Call Timing

- Called at object creation
- Executes before any function call
- Student s1(10);

```
#include <iostream>
using namespace std;
 class Student {
private:
   int roll;
 public:
   Student(int r) { roll = r;   }
   void display() {  cout << "Roll number: " << roll << endl;    }
};
int main() {
   Student s1(40);
   s1.display();
   return 0;
}
```

- Object initialized at creation
- Constructor ensures valid state

```
#include <iostream>
using namespace std;
class Car {
private:
   string brand;
   int speed;
public:
   Car() {// Default constructor
      brand = "Toyota";
      speed = 120;  }
   void display() {
      cout << "Brand: " << brand << endl;
      cout << "Speed: " << speed << " km/h" << endl;
   }
};
```

```
int main() {
   Car c1;     // constructor automatically called
   c1.display();
   return 0;
}
```

```
#include <iostream>
using namespace std;
class Student {
private:
   string name;
   int marks;
public:
   // Parameterized constructor
   Student(string name1, int marks1)
{      name = name1; marks = marks1;
}
   void result() {
      cout << "Name: " << name << endl;
      cout << "Marks: " << marks <<
endl;
   }
};
```

```
int main() {
   Student s1("Arun", 95);
   Student s2("Meena", 88);
   s1.result();
   cout << endl;
   s2.result();
   return 0;
}
```

What is a Destructor

- Special member function
- Name same as class with ~
- No parameters
- No return type

Destructors

- Clean up object resources
- Called automatically
- End of object lifecycle

Destructor Syntax

- ~Student() {
- cout << "Object destroyed" << endl;
- }

```
#include <iostream>
using namespace std;
class Student {
public:
   Student() {       cout << "Constructor called" << endl; }
   ~Student() {       cout << "Destructor called" << endl;  }
};
int main() {
   Student s1;
   return 0;
}
```

- Shows object creation
- Shows object destruction
- Demonstrates lifecycle clearly

Problem of Name Conflict

- Data member and parameter can have same name
- Creates ambiguity inside member function
- Assignment may not work as expected

```
class Student
{
Int roll;
void setRoll(int roll) {
   roll = roll;   // ambiguous
}
};
```

Why this Pointer is Needed

- Refers to the current calling object
- Distinguishes data member from parameter
- Resolves naming conflicts

What is ‘this’ Pointer

- Implicit pointer available in member functions
- Stores address of current object
- Used to access data members explicitly
-

Using this Pointer

- void setRoll(int roll) {
- this->roll = roll;
- }
- this->roll → data member
- roll → function parameter

```
#include <iostream>
using namespace std;
class Student {
private:
   int roll;
public:
   Student(int roll) {   // same name
      roll = roll;      // PROBLEM LINE
       }
 void display() {         cout << "Roll number: " << roll << endl;      }
};
int main() {
   Student s1(55);
   s1.display();
   return 0;
}
```

```
#include <iostream>
using namespace std;
class Student {
private:
   int roll;
public:
   Student(int roll) {       this->roll = roll;    }
   void display() {       cout << "Roll number: " << roll << endl;    }
};

int main() {
   Student s1(55);
   s1.display();
   return 0;
}
```

- Constructor parameter and data member have same name
- this resolves ambiguity

SCOPE RESOLUTION OPERATOR (::)

- Used to define class members outside class
- Resolves scope-related ambiguity
- Improves code organization
- Commonly used for member functions

Why Scope Resolution Operator is Needed

- Keeps class definition clean
- Separates interface and implementation
- Improves readability in large programs

Syntax of Scope Resolution Operator

```
return_type ClassName::functionName() {
   // function body
}
```

- :: connects function to class
- Function defined outside class

- Instead of writing:

```
class Student {
public:
   void display() { cout << "Hello"; }
};
```

- We do this:

```
class Student {
public:
   void display();  // declaration
};
void Student::display() {   // definition
   cout << "Hello";  }
```

```
#include <iostream>
using namespace std;
class Student {
private:
   int roll;
public:
   void setRoll(int r);
   void display();
};
void Student::setRoll(int r) {
   roll = r;
}
void Student::display() {
   cout << "Roll number: " << roll << endl;
}
```

```
int main() {
   Student s1;
   s1.setRoll(70);
   s1.display();
   return 0;
}
```

- Class declared first
- Functions defined outside using ::
- Clean separation of definition and implementation

Static Data Members

- Belong to the class, not to objects
- Shared among all objects
- Only one copy exists

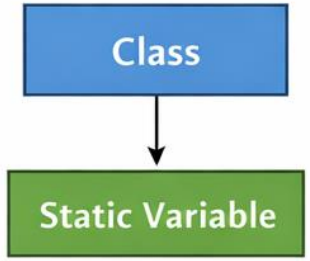
Why Static Data Members Are Needed

- To maintain common data
- To count number of objects
- To share information across objects

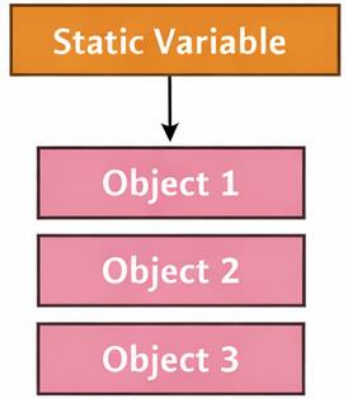
Static vs Non-Static Data Members

- Non-static → separate copy per object
- Static → single shared copy
- Accessed using class name or object

Static Data Member

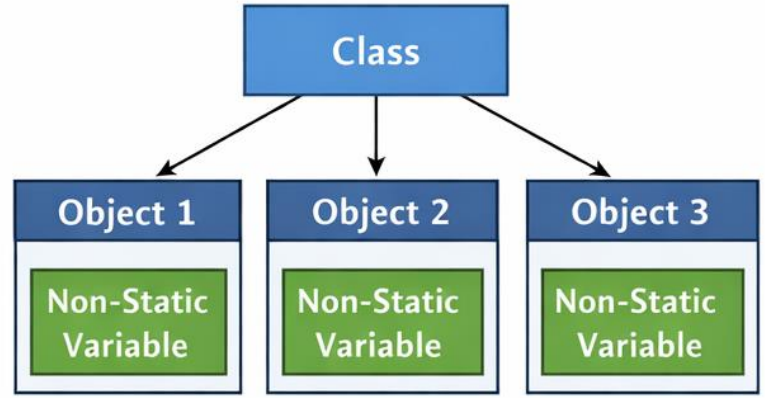


Memory (Class Level):

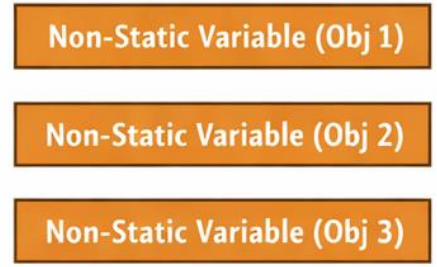


No Copies in Objects

Non-Static Data Member



Memory (Object Level):



Separate Copies in Each Object

Syntax of Static Data Member

```
class Counter {
public:
    static int count;
};
```

- Declared inside class
- Defined outside class

Defining Static Data Member

```
int Counter::count = 0;
```

- Memory allocated once
- Must be defined outside class

```
#include <iostream>
using namespace std;
class Counter {
public:
   static int count;
   Counter()
      {  count++; }
   void show()
   {
     cout << "Count: " << count << endl;
   }
};
```

```
int Counter::count = 0;
int main() {
   Counter c1, c2, c3;
   c1.show();
   c2.show();
   c3.show();
   return 0;
}
```

- All objects share same count
- Demonstrates class-level data

Static Data Member – Memory Behaviour

- Exists even if no objects are created
- Created once for the class
- Destroyed at program termination

Static Data Member with Non-Static

```
#include <iostream>
using namespace std;
class Sample {
public:
   static int x;
   int y;
  Sample(int a, int b) {
      x = a;
      y = b;
   }

void display() {
  cout << "x: " << x << " y: " << y << endl;
   } };
```

```
int Sample::x = 0;
 int main() {
   Sample s1(10, 20);
   Sample s2(30, 40);
   s1.display();
   s2.display();
   return 0;
}
```

- x is shared
- y is separate for each object

Key Difference – Static vs Non-Static

- Static → common to all objects
- Non-static → unique per object
- Use static only when sharing is required

Static Data Members – Summary

- Declared inside class
- Defined outside class
- Shared across all objects
- Useful for counters and shared data

Static Member Functions

- A **static member function** is a function that belongs to the **class itself**, not to any object.
- Normal functions → work with objects
  Static functions → work with the class
- Example:

```
class Demo {
public:
   static int x;
   static void show() {
      cout << x;
   } };
```

<u>Call</u>

```
Demo::show();
```

Static Member Functions

- Static data often accessed via functions
- Static member functions belongs to the class
- Can access only static members
- Do not depend on object
- Provide controlled access
- Improve design clarity
- Can be called using class name
- Cannot access non-static members directly
- Do not use this pointer

Why Static Member Functions?

- **Without static member function:**
  - Counter::count++; // external modification
- **With static member function (better):**
  - Counter::increment(); // controlled modification
  - Counter::display(); // controlled reading
- So static member functions are like **utility tools attached to the class**.

```
class A {
public:
    static int x;
    int y;
    static void showX() { cout << x; } // Is this correct?
    static void showY() { cout << y; } // Is this correct?
};
```

```
#include <iostream>
using namespace std;
class Counter {
public:
    static int count;
    static void increment()
                {       count++;    }
    static void display() {
        cout << "Count: " << count << endl;
    }
};
```

```
int Counter::count = 0;
int main() {
    Counter::increment();
    Counter::increment();
    Counter::display();
    return 0;
}
```

- Function accessed using class name
- Operates only on static data

When to Use Static Member Functions

- When logic is class-level
- When object is not required
- For shared behaviour

const MEMBER FUNCTIONS

- A **const member function** is a class function that **promises NOT to modify the object's data members**.
- It tells the compiler:
- "I will only READ the object, I will not CHANGE it."
- Prevent modification of object data
- Enforce read-only behaviour
- Improve program safety

Meaning of const After Function

- Guarantees function will not modify data
- Compiler enforces restriction
- Syntax
  - return_type function_name() const;

- void display() const;

Rules of const Member Functions

- Cannot modify data members
- Can call only other const functions
- Can be called on const objects

```
#include <iostream>
using namespace std;
class Student {
private:
   int roll;
public:
   Student(int r) {
      roll = r;
   }
   void show() const {
      cout << "Roll: " << roll << endl;
   } };
int main() {
   Student s1(10);
   s1.show();
}
```

const Member Functions – Summary

- Protect object state
- Enforced at compile time
- Useful for read-only operations

Inline function

- An **inline function** is a function where the compiler replaces the function call with the actual function code.
- Instead of:
  - calling the function → jumping → returning
  - the compiler directly **copies the function body** into the program.

Normal Function (usual)

```
int add(int a, int b)
{   return a + b; }
int main()
{   int c = add(2,3); }
```

main() → go to add() → calculate → return → come back

This jumping is called **function call overhead**
(time + stack memory is used)

Inline Function

```
inline int add(int a, int b)
{    return a + b; }
```

<u>Compiler changes from this</u>

```
int c = add(2,3);
```

<u>To this</u>

```
int c = 2+3;
```

(No function call happens!)

Why Inline Functions are Used

- To make **small, frequently used functions faster**.
- It improves:
  - execution speed
  - performance
- Especially useful in:
  - mathematical calculations
  - getters/setters
  - small operations

<u>Syntax</u>

```
inline return_type function_name(parameters)
{
   // function body
}
```

```
#include <iostream>
using namespace std;
inline int square(int x)
{
   return x * x;
}
int main()
{
   cout << square(5);
   return 0;
}
```

No → square(5);
Yes → 5*5

Inline is only a REQUEST

- The compiler may **ignore inline** if:
  - function is large
  - contains loops
  - contains recursion
  - contains switch or static variables
- So:
- inline does not force — it only suggests.

Normal Function	Inline Function
Function call happens	No function call
Slower	Faster
Less memory	More memory
Good for big code	Good for small code

Friend

- In C++: private and protected members are **hidden** from outside classes.
- But sometimes we need a **special outside function or another class** to access private data.
- C++ provides a keyword:
  - friend
- A *friend* is **NOT a member**, but it is allowed to access private data.

Friend function

- A **friend function** is a normal function that is permitted to access the private and protected members of a class.
- It is declared inside the class using the keyword friend.
- **Syntax**

```
class ClassName {
private:
        int data;
public:
        friend void functionName(ClassName obj);
        };
```

- It is declared inside the class
- But defined outside the class
- Not a member function

```
#include <iostream>
using namespace std;
class Box {
private:
   int length;
public:
   Box(int l) { length = l; }
   void showLength() {        // ✅ member function
      cout << "Length = " << length << endl;   // ✅ allowed
   } };
int main() {
  Box b(10);
  b.showLength(); }
```

```
#include <iostream>
using namespace std;
class Box {
private:
   int length;
public:
   Box(int l) { length = l; }
};
void showLength(Box b) {        // ❌ NOT a member, NOT a friend
   cout << b.length << endl;  }  // ❌ ERROR: private member access
int main() {
   Box b(10);
   showLength(b); }
```

```
#include <iostream>
using namespace std;
class Box {
private:
   int length;
public:
   Box(int l) { length = l; }
   friend void showLength(Box b);   // ✅ friend declaration
};
void showLength(Box b) {          // ✅ friend function (not a member)
   cout << "Length = " << b.length << endl;   // ✅ allowed
}
int main() {
   Box b(10);
   showLength(b);
}
```

Friend class

- A **friend class** means:
- One entire class is allowed to access all private and protected data members of another class.
- So instead of giving permission to **one function**, we give permission to **the whole class**.
- **Syntax**

```
class A {
   friend class B;
   };
```

- This means:
- **Every function inside class B can access private members of class A.**

```
#include <iostream>
using namespace std;
class A {
private:
   int secret;
public:
   A() { secret = 50; }
   // Entire class B is friend
   friend class B;
};
class B {
public:
   void show(A obj) {
      cout << "Secret: " << obj.secret << endl;
   }
   void change(A &obj) {
      obj.secret = 200;
  }};
int main() {
   A a1;
   B b1;
   b1.show(a1);   // access private data
   b1.change(a1); // modify private data
   b1.show(a1);
}
```

Lambda Expressions

- A Lambda expression is an anonymous (nameless) function defined right inside another function.

- Regular usage vs one time usage?

- Lambda is useful when the logic is:
  - short,
  - used only once,
  - and mostly used with STL algorithms (sort, count_if, etc.).

Basic Syntax

- Syntax
- **[](){};**
- [Capture] (Parameters) { Body };
- [capture](parameters) -> return_type { function body};

```
[](int x, int y) { return x + y; };
```

- **(A) Capture clause [ ]**
- Tells what external variables the lambda can use.
- **(B) Parameters ( )**
- Same as function parameters.
- **(C) Return type -> type (optional)**
- Usually compiler automatically deduces it.
- **(D) Body { }**
- Function logic.

Why Do We Need Them?

- The Issue: Before C++11, even for a tiny 1-line logic, you had to write a full function outside main(). This scatters your logic across the file.
- The Solution: Lambdas keep the logic exactly where it is used, making the code "locally readable."
- Example: If you want to compare two numbers just once in your whole program.

- <u>Comparison Code:</u>

```
// OLD WAY: Must define this outside main()
bool compare(int a, int b) { return a > b; }

int main() {
   // NEW WAY: Defined exactly where needed
   auto myLambda = [](int a, int b) { return a > b; };
}
```

- A lambda is not just “a function”.
  It is an **object that behaves like a function** (function object).
- So you can store it:
- auto f = [](int x){ return x % 2 == 0; };
  cout << f(10); // calls it like a function

- Output - 1

```
#include <iostream>
using namespace std;
int main() {
    // Storing a lambda in a variable
    auto multiply = [](int a, int b) {return a * b; };
    int result = multiply(5, 4); // Used just like a function call
    cout << "Product: " << result; // Output: 20
    return 0;
}
```

Why do we need Lambda Expressions?

- **Problem in old C++ (before C++11)**
- When you want to pass “logic” into a function (like sorting rule), you had to:
  - write a separate function, or
  - create a separate class.
- That makes code long and messy for small rules.
- Example: sort descending
  Old way:
- bool desc(int a, int b) { return a > b; }
  sort(v.begin(), v.end(), desc);
- **Lambda way (clean)**
- sort(v.begin(), v.end(), [](int a, int b){ return a > b; });
- So lambdas reduce boilerplate and improve readability.

3 General Issues

- **A. Name Pollution -** For every small rule (like isEven, isOdd, descending, greaterThanTen), you had to give it a unique name. In a large project, you end up with hundreds of tiny functions like compare1, compare2, mySortRule, which makes the code hard to manage.
- **B. Boilerplate (The "Functor" Nightmare) -** If your sorting rule needed to stay flexible (e.g., "Sort by a limit that the user types in"), a simple function wouldn't work because functions can't "store" state. You had to create a **Class** with an overloaded operator().

```
class Comparator {
    int limit;
public:
    Comparator(int l) : limit(l) {}
    bool operator()(int a, int b) { return a > limit; }
};
// You had to write all this just to compare two numbers!
```

- **C. Distance -** When a programmer reads your code, they have to keep jumping up and down the file to see what desc or myRule actually does. This breaks their "flow" of understanding.

Lambda Way to Solve these issues

- A Lambda is "Anonymous"—it has no name. It exists only for that specific line of code.
- C++
  - sort(v.begin(), v.end(), [](int a, int b){ return a > b; });
- Why it reduces "Boilerplate":
- No Name Needed: You don't have to invent a name like compareDescending.
- No Class Needed: The compiler creates the "class/functor" for you automatically behind the scenes.
- Locality: The logic a > b is right there inside the sort call. Anyone reading the code immediately knows the sorting rule without scrolling.

Lambda Expression vs Function

Feature	Normal Function	Lambda Expression
Name	Yes	No (anonymous)
Reuse	Easy	Mostly for one-time use
Capture local variables	No	Yes
Used with STL algorithms	Less convenient	Very convenient

Capture [ ]

- Lambda can “use” variables from the surrounding function. This is the *special power*.

(A) Capture nothing []

Lambda cannot use outside variables.

```
int x = 10;
auto f = [](){ /* cannot use x here */ };
```

(B) Capture by value [x]

Copies the value into lambda.

```
int x = 10;
auto f = [x]() {cout << x;};
f();   // prints 10
```

If x changes later, lambda still has old copy.

(C) Capture by reference [&x]

Uses original variable (can modify it).

```
int x = 10;
auto f = [&x]() {x = x + 5;};
f();
cout << x;  // 15
```

(D) Capture all by value [=]

- All external variables used inside are copied.

```
int a = 2, b = 3;
auto f = [=]() { return a + b; };
a=10;b=20;
cout<<f()<<a<<b;
```

(E) Capture all by reference [&]

All external variables used inside are referenced.

```
int a = 2, b = 3;
auto f = [&]() { a++; b++; };
```

(F) Mixed capture

```
int a = 10, b = 20;
auto f = [a, &b]() {
   // a copied, b referenced
   b = b + a;
};
```

```
#include <iostream>
using namespace std;
int main() {
    int milk = 10;
    auto photo = [milk]() { cout << "Photo: " << milk << endl; };
    auto original = [&milk]() { milk = 20; };
    milk = 15; // Change the original
    photo();   // Still prints 10 (The photocopy didn't change)
    original(); // Changes the original to 20
    cout << "Final Milk: " << milk << endl; // Prints 20
    return 0;
}
```

STL

- **STL = Standard Template Library** in C++.
- It is a ready-made library that gives:
- **Containers**: vector, list, deque, set, map, etc.
- **Algorithms**: sort, find, count_if, etc.
- **Iterators**: a standard way to traverse containers.
- **Utilities**: pairs, function objects, etc.
- STL is a collection of pre-built data structures and algorithms provided by C++.
- To access STL Lambda expressions are used.

```
#include <iostream>
#include <vector>
#include <algorithm> // Required for STL Algorithms
using namespace std;
int main() {
   vector<int> numbers = {12, 5, 8, 20, 1};
   // 1. Sort the numbers in DESCENDING order using a Lambda
   sort(numbers.begin(), numbers.end(), [](int a, int b) {return a > b; });
   cout << "Sorted: ";
   for(int n : numbers) cout << n << " "; // Output: 20 12 8 5 1
   // 2. Count numbers greater than 10
   int count = count_if(numbers.begin(), numbers.end(), [](int n) {return n > 10;});
   cout << "\nNumbers > 10: " << count; // Output: 2
   return 0; }
```

Return Type in Lambda

- Usually auto deduced.
  But when your lambda has multiple return paths, specify it:

```
auto f = [](int x) -> double {
   if (x > 0) return x * 1.5;
   else return 0.0;
};
```

Lambdas with STL

(A) sort with custom rule

```
vector<int> v = {4, 1, 7, 2};
sort(v.begin(), v.end(), [](int a, int b){
    return a > b;  // descending
});
```

(B) count_if

Count even numbers:

```
int c = count_if(v.begin(), v.end(), [](int x)
{    return x % 2 == 0; });
```

(C) for_each

```
for_each(v.begin(), v.end(), [](int x)
{ cout << x << " "; });
```

Lambda creates a “function object”

- Technically, lambda is not a normal function.
- It creates an object of a hidden compiler-generated class with operator().
- Conceptually:

```
class Hidden {
public:
   int operator()(int a, int b) const {
      return a + b;
   }
};
```

That’s why:

- lambda can be stored in variables using auto
- lambda can capture values (store them in the object)
- This captured object is often called a **closure**

“mutable” in lambda

- By default, if you capture by value, you cannot modify it inside the lambda.

```
int x = 10;
auto f = [x]() { x++; }; // error
```

- Use mutable:

```
int x = 10;
auto f = [x]() mutable { x++; cout << x; };
 f(); // prints 11
 cout << x; // still 10 outside (because it was copied)
```

When to use Lambda?

- Use lambda when:
  - logic is short
  - used only once
  - used inside sort/search/filter
  - you need to use local variables inside rule
- Avoid lambda when:
  - logic is large and complex
  - used in many places (better to use normal function)